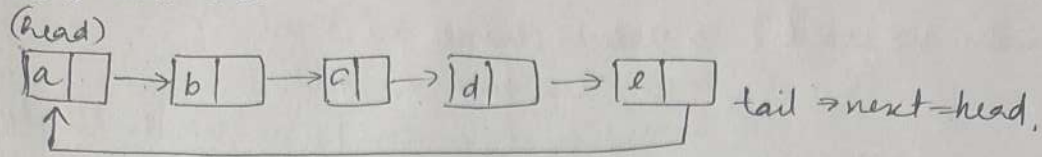


Circular Linked List :

→ Last node ka next → points to the head of the list.

→ Null kabhi nahi hota



① Traversal : Traversal of infinite loop, by :

```
void printll(Node* head)
```

```
    if (head == NULL) return;
```

```
    Node* temp = head;
```

```
    do {
```

```
        cout << temp->val << " ";
```

```
        temp = temp->next;
```

```
    } while (temp != head);
```

// This loop ensure 1 extra iteration

② Insert at beggining :

```
Node* insertathead(list* head, int x)
```

```
    Node* temp = new Node(x); // creation
```

```
    if (!head || head == NULL)
```

```
    {
        temp->next = temp;
```

// self joining // single node

```
        return temp;
```

```
    Node* curr = head;
```

```
    while (curr->next != head)
```

[// move to tail]

```
    {
        curr = curr->next;
```

[// keep traversing]

```
    curr->next = temp;
```

return temp;

```
    temp->next = head;
```

③ Insert at head $O(1)$ Trick

Node * insertathead (Node * head, int x)

Node * temp = new Node(x);

if (head == NULL)

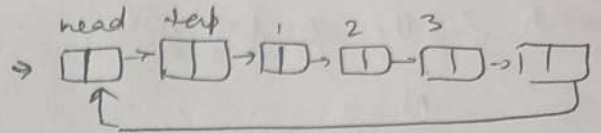
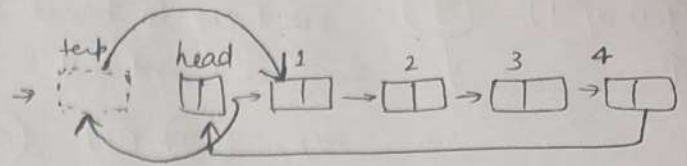
temp = temp → next;
return temp;

temp → next = head → next;

head → next = temp;

swap (head → val, temp → data);

return head;



→ now, swap data of both.

Insert at beginning without data swap

insertathead (list * head, int x)

list * temp = new list(x);

if (head == NULL)

temp → next = temp;
return temp;

list * cur = head;

while (cur → next)

cur = cur → next;

~~temp → next = cur → next;~~

cur → next = temp;

temp → next = head;

with data swap

insertathead (list * head, int x)

list * temp = new list(x);

if (head == NULL)

temp → next = temp;
return temp;

// NO linking, head, remains head;

temp → next = head → next;

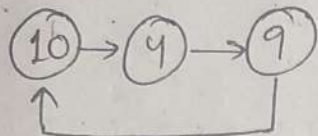
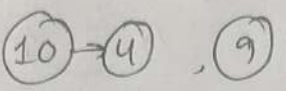
head → next = temp;

swap (head, temp);

return head;

QFG

Split a linked list into 2 halves

I/p.  , we have to break this linked list into two
~~Singular~~ ^{Circular} linked list
o/p, 

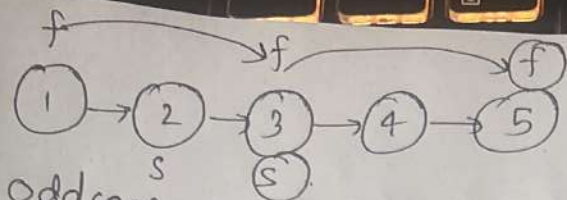
Approach, find the start of cycle.

```
slow = S → N;  
fast = F → N → N;  
if (slow == fast) break;  
if (S != F) S → N, F → N;
```

① head → mid

② mid → next → end

```
Code : pair<Node*, Node*> splitList(Node* head)  
{  
    if (!head || head->next == head)  
        return (head, nullptr);  
  
    Node* slow = head;  
    Node* fast = head;  
    while (fast->next != head + f->n->n)  
    {  
        slow = slow->next;  
        f = f->n->n;  
    }  
    if (f->n->n == head) f = f->n;  
  
    Node* head1 = head;  
    Node* head2 = S->n;  
  
    slow->next = head1;  
    fast->next = head2;  
    return {head1, head2};  
}
```



Odd case

while($f \rightarrow n \neq \text{head}$ + $f \rightarrow n \rightarrow n \neq \text{head}$)

✖

$S = S \rightarrow N;$

$f = f \rightarrow N \rightarrow N;$

✖

if ($f \rightarrow n \rightarrow n \neq \text{head}$) $f = f \rightarrow N;$

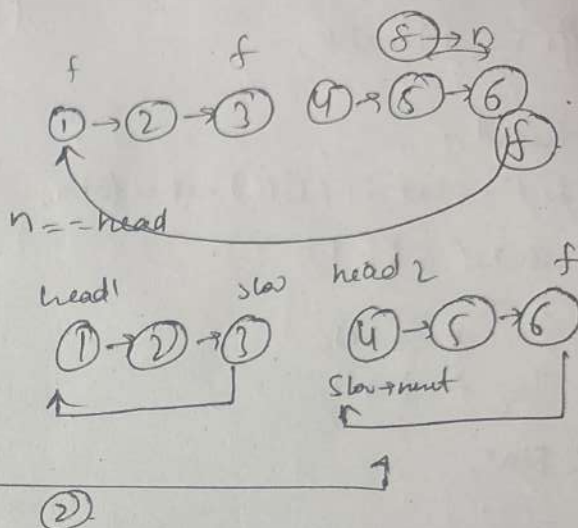
Node* head1 = head;

Node* head2 = slow → next;

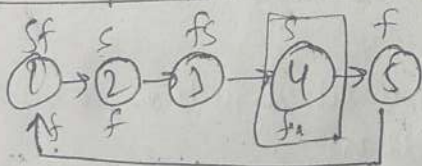
slow → next = head1

$f \rightarrow \text{next} = \text{head2}$

return {slow, fast};



Check if a Linked list is circular



→ Use slow / fast pointer to detect cycle also known as Floyd's Cycle detection.

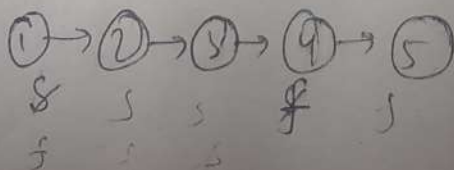
→ $S = S \rightarrow N$, $f = N \rightarrow N$, if (both are equal at certain point)

→ cycle exist in this case.

→ at end, cycle exist,

→ if not, then return false, out of while loop.

→ and start of cycle is (slow → next)



```
Node* slow = head;
    fast = head;
```

```
while (fast && fast->next)
```

```
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) break;
```

```
slow = head;
```

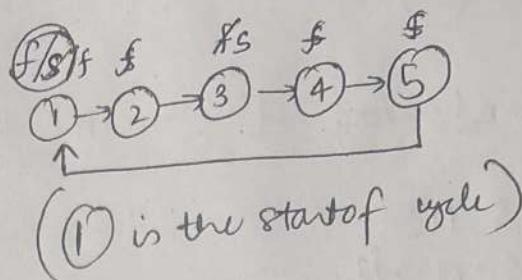
```
if (fast->next == NULL) return false;
```

```
else while (slow->next == fast->next->next)
```

```
    slow = slow->next;
```

```
    fast = fast->next->next;
```

```
return slow;
```



cycle detection &
Circular linked list detection
both are different;

(Remove duplicates from LL)

I/P 1 → 2 → 2 → 3 → 3 → 4 → 5

1 → 4 → 5

(with same duplicate values as well as for nodes)
sb hatana hai idhar.

```
Node* dummy(0);
```

```
dummy->next = head;
```

```
prev = dummy;
```

```
curr = head;
```

```
duplicate = false;
```

```
while (curr)
```

```
    if (false)
```

```
        while (curr->next && curr->val == curr->next->val)
```

```
            duplicate = true;
```

```
            curr = curr->next;
```

```
        }
```

if (duplicate)

 prev → next = cur → next; } [dummy → next → next = cur → next]
else {
 prev = prev → next;
 cur = cur → next;
}

(meaning)

return dummy → next;

Approach ⇒ dummy kuse karna hai

→ prev ko dummy mai rakho.

→ cur = head

→ iterate through (head)

dup = false;

while (cur → next && cur → val == cur → next → val)

 dup = true;

 cur = cur → next;

if (dup)

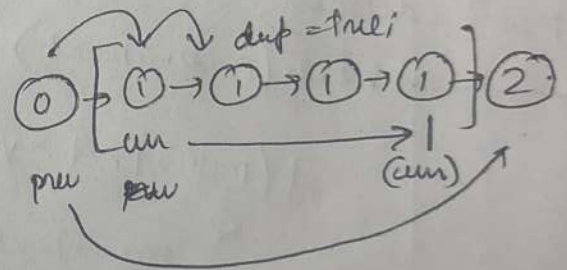
 prev → next = cur → next;

else

 prev = prev → next;

cur = cur → next;

return dummy → next;

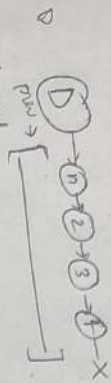


Dummy / in Place reversal

Dummy → node at temporary placeholder, is linked list & start main, add kya jata hai,

→ This method helps us to modify list without modifying its head.

→ Dummy(0). Value doesn't matter, only what matters is created before head, (dummy.next = head)



Delete node by val.

method dummy(0);

dummy.next = head;

prev = &dummy;

~~if head value (prev → next)~~

if (prev → next → val == N)

prev → next = prev → next → next;

else { prev = prev → next;

return dummy → next;

✓

IN-Place reversal → may use prev, curr

→ save curr → next into next

→ prev → next = curr

We change the existing pointer without even changing/creating new links

① Dummy Node method

Pros: head & south special one handle karna easy hai

→ reversible easy to read

→ useful for complex linked list manipulations

Used when:

① head to modify karna risky hai

② Multiple insertion / deletion

③ clean & safe code

② In-place reversal

- No extra memory $O(1)$

- fast / efficient

- directly linked list ko reverse krta hai

* Used when extra list ya subset ko reverse krna hai

→ Memory optimization important

→ simple & efficient solution

eg Reverse a linked list (Sublist) $m=2, n=4$ $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

if $p \rightarrow [1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5]$

or $p \rightarrow [1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5]$

dummy = 0;

dummy.next = head;

prev = &dummy;

for (i = 1; i < m; i++)

prev = prev → next;

curr = prev → next; // yha karna hai

next = NULL;

for (i = 0; i < n - m; i++) curr → next = prev → next → next;

next = curr → next;



next = curr → next; // save

curr → next = next → next;

next = curr → next;

curr → next = next → next;

next = curr → next;